Documento de Arquitectura - Lado del Cliente

1. Contexto del proyecto

Nombre y propósito del sistema: "Gestor de Tareas Académicas". Es una aplicación web diseñada para ayudar a los estudiantes universitarios a organizar, priorizar y realizar un seguimiento de sus actividades académicas (tareas, proyectos, exámenes).

Problema que busca solucionar: La carga académica universitaria suele ser abrumadora. Los estudiantes frecuentemente olvidan fechas de entrega o tienen dificultades para organizar sus pendientes entre diferentes asignaturas, lo que genera estrés y afecta el rendimiento académico.

Usuarios principales: Estudiantes universitarios (principalmente de ingeniería y desarrollo de software, pero adaptable a cualquier facultad) que buscan una herramienta rápida y sin distracciones.

Funcionalidades más importantes:

- Creación rápida de tareas o recordatorios.
- Marcado de tareas como completadas/pendientes.
- Eliminación de tareas obsoletas.
- Filtrado de vistas (Todas, Pendientes, Completadas).
- *Visualización de estadísticas de progreso académico (funcionalidad proyectada).*
- *Configuración de preferencias del usuario (funcionalidad proyectada).*

Información que necesita gestionar:

- **Tareas:** Identificador único, descripción de la actividad y estado de completitud.
- **Usuario:** Preferencias de la interfaz (tema claro/oscuro) y datos básicos de perfil (ej. semestre, carrera).

---

2. Decisión sobre cómo entregar la aplicación

Elección: SPA (Single Page Application)

Para la construcción y entrega de la interfaz hemos seleccionado el enfoque SPA utilizando la librería React junto con Vite como empaquetador.

Justificación y adecuación al proyecto: Dado que el Gestor de Tareas es una herramienta interactiva de uso continuo, el usuario requiere respuestas instantáneas al agregar o completar tareas. Una SPA descarga un único documento HTML y los archivos JavaScript necesarios al inicio; a partir de ahí, la interfaz se actualiza dinámicamente en el navegador sin recargar la página. Al ser una herramienta de productividad de acceso privado (requerirá inicio de sesión a futuro), no dependemos de la indexación en motores de búsqueda (SEO), lo cual es el principal punto débil de las SPA.

Ventajas:

- **Experiencia de usuario (UX) fluida:** Transiciones rápidas y sensación de estar usando una aplicación nativa.
- **Menor carga en el servidor:** Una vez descargado el cliente, la aplicación gestiona la UI internamente y solo se comunicaría con un servidor mediante APIs (o en nuestro caso actual, con el LocalStorage).

Limitaciones:

- **SEO pobre:** Al generarse el contenido con JavaScript, los rastreadores web tienen dificultades para indexar la página.
- **Carga inicial (First Contentful Paint):** El tiempo de carga inicial puede ser ligeramente mayor mientras se descarga el *bundle* de JavaScript.

---

3. Organización interna del cliente

El código del lado del cliente está organizado mediante una **arquitectura basada en componentes y funcionalidades cruzadas**, una convención estándar en el ecosistema React.

Estructura de carpetas propuesta:

```
src/
 ├── assets/       # Imágenes, íconos y recursos estáticos.
 ├── components/   # Componentes de UI reutilizables (Botones, Tarjetas, Inputs).
 ├── pages/        # Componentes a nivel de página (Vistas enrutables).
 ├── hooks/        # Lógica de estado encapsulada (ej. useLocalStorage).
 ├── App.jsx       # Componente raíz que orquesta las rutas y el layout global.
 └── index.css     # Estilos globales y variables CSS.
```

Justificación: Esta separación de responsabilidades permite que el proyecto escale de manera ordenada. La carpeta `components` aloja piezas puramente visuales y aisladas, mientras que `pages` aloja componentes más complejos que representan pantallas completas y se conectan a los estados globales. La lógica de persistencia se abstrae en `hooks`, evitando acoplar la interfaz con la forma en que se guardan los datos.

---

4. Componentes y rutas

Para darle mayor robustez al sistema, la aplicación cuenta con un sistema de enrutamiento del lado del cliente (ej. usando `react-router-dom`).

Rutas Principales

- `/` **(Dashboard / Inicio):** Es la pantalla principal donde el usuario visualiza, agrega y filtra sus tareas diarias.
- `/perfil` **(Perfil del Estudiante):** Pantalla dedicada a mostrar un resumen de la productividad del estudiante (ej. tareas completadas por semana) e información de su semestre.
- `/configuracion` **(Ajustes):** Pantalla para gestionar opciones de la aplicación, como la limpieza total de datos o el cambio de tema visual.

Componentes Principales

1. `TaskForm`: Renderiza el formulario de entrada. *Responsabilidad:* Capturar la entrada de texto del usuario y emitir un evento para crear una tarea. *Reutilizable:* Sí.
2. `TaskList`: Renderiza el contenedor de la lista de tareas. *Responsabilidad:* Recibir un arreglo de tareas e iterarlo para renderizar los elementos individuales. *Reutilizable:* Sí (podría usarse para listas de proyectos a futuro).
3. `TaskItem`: *Responsabilidad:* Representar visualmente una sola tarea, incluyendo los botones de "Completar" y "Eliminar". *Reutilizable:* Sí.
4. `Filters`: *Responsabilidad:* Mostrar botones de navegación entre estados (Todos, Pendientes, Completados) y cambiar la vista activa.

5. `Navbar` : *Responsabilidad:* Barra de navegación superior para moverse entre el Dashboard, el Perfil y la Configuración. *Reutilizable:* Sí (se usa en todas las rutas).

---

5. Datos y estado de la aplicación

La aplicación gestiona su estado predominantemente en la memoria del navegador y se apoya en el **LocalStorage** para la persistencia de datos.

Datos principales manejados:

1. Colección de tareas (Array de objetos con `id` , `text` , y `completed`).
2. Filtro activo actual (String: 'all', 'pending', 'completed').
3. Preferencias del usuario (Tema oscuro/claro).

Flujo y compartición de la información:

- **Compartición entre componentes:** La lista maestra de tareas se mantiene en un estado alto en la jerarquía (ej. en el componente página `Dashboard` o a través del `App.jsx`). Estos datos fluyen "hacia abajo" mediante *props* hacia `TaskList` y `Filters` .
- **Información al navegar entre rutas:** Dado que la aplicación permite navegar a `/perfil` (donde se necesitan contar las tareas completadas) y a `/configuracion` , el estado de las tareas debe sobrevivir a la navegación. Esto se logra centralizando el acceso al `LocalStorage` a través de un *Custom Hook* (`useLocalStorage`), el cual es consumido por las distintas páginas cuando necesitan leer o modificar la información persistida.

---

6. Diagrama de arquitectura

A continuación se muestra el diagrama arquitectónico que representa la entrega mediante SPA, las rutas, los componentes internos y su comunicación con el almacenamiento local.

```
architecture-beta
    group client(internet)[Navegador Web - Cliente SPA]

    service app(server)[App.jsx / Enrutador] in client
    service page1(database)[Ruta: /] in client
    service page2(database)[Ruta: /perfil] in client
    service page3(database)[Ruta: /configuracion] in client

    service comp1(disk)[TaskForm] in client
    service comp2(disk)[TaskList] in client
    service comp3(disk)[TaskItem] in client

    service storage(disk)[LocalStorage (Persistencia)] in client

    app:R --> L:page1
    app:R --> L:page2
    app:R --> L:page3

    page1:B --> T:comp1
    page1:B --> T:comp2
    comp2:B --> T:comp3
```

```
page1:L --> R:storage
page2:L --> R:storage
```